System Architecture

Source: docs/architecture.md

System Architecture

Status: internal technical documentation Last updated: 2026-05-23 Extended reference: `SYSTEM_OVERVIEW.md`, `ARCHITECTURE_DEEP_DIVE.md`

Overview

The platform is a React + FastAPI application for ESG document ingestion, RAG retrieval, background report generation and PDF export.

```
[text]
React 19 / Vite
    |
    | HTTP REST, JWT bearer auth
    v
FastAPI backend
    |
    | Celery tasks
    v
Redis broker/result backend + Celery worker/beat
    |
    +--> OpenAI embeddings: text-embedding-3-small
    +--> OpenAI chat/report generation: gpt-4o-mini
    +--> Supabase/PostgreSQL + pgvector
    +--> PostgreSQL access through database/* repositories
```

Frontend

- React 19, Vite and React Router DOM.
- API base URL comes from `VITE_API_URL`; fallback is `http://localhost:8000`.
- Optional report model badge comes from `VITE_REPORT_MODEL_LABEL`; fallback is `AI POWERED`.
- `frontend/src/components/MultiFileUpload.jsx` handles multi-file uploads, per-file tags and polling `/status/{task_id}`.
- `frontend/src/pages/Dashboard.jsx` lists user documents and starts report generation for `ESG`, `Environmental`, `Social` or `Governance`.
- `frontend/src/pages/AIReports.jsx` starts `/report/generate`, polls status and downloads `/report/download/{task_id}` as a PDF blob.

Backend

- FastAPI application entrypoint: `backend/main.py`.
- Authentication: JWT through `backend/auth.py`, with users stored in `app_users`.
- Background tasks: Celery configuration in `backend/celery/celery_app.py`.
- Report generation: `backend/celery/report_tasks.py`.
- Chat and ingestion tasks: `backend/celery/tasks.py`.
- RAG retrieval: `backend/RAG/rag_retriever.py`, using Supabase RPC `match_chunks2`.
- PDF rendering: `backend/utils/pdf_generator.py`, using ReportLab.

Queues

Queue	Purpose
`default`	General background tasks
`parsing`	Document parsing and ingestion
`embeddings`	Single-document embedding work
`embeddings_bulk`	Bulk embedding generation
`llm`	Chat and report LLM calls

Redis is used for Celery broker/result storage. The backend also stores task ownership in Redis for `/status/{task_id}` and `/report/download/{task_id}`.

Data Storage

Active Supabase/PostgreSQL tables used by the current code:

- `app_users`
- `user_documents`
- `user_document_chunks`
- `knowledge_documents`
- `knowledge_chunks`
- `reports`
- `chat_sessions`
- `chat_messages`

The vector chunks live in `user_document_chunks` and `knowledge_chunks`, both with pgvector embeddings. Report history is stored in `reports`; `used_chunks` is stored as JSON text.

Report Pipeline

- Frontend posts `{ "report_scope": "Environmental" | "Social" | "Governance" | "ESG", "standard": "GRI" | "SASB" | "TCFD" }` to `/report/generate`.
- FastAPI queues `backend.generate_report` and registers task ownership.
- Celery retrieves chunks through `match_chunks2`.
- Partial scopes try tag aliases only; `ESG` retrieves without a tag filter.
- The report task separates company chunks from legal/knowledge-base chunks.
- OpenAI `gpt-4o-mini` returns structured JSON aligned to the selected standard checklist.
- The JSON and `used_chunks` are saved to report history when possible and

returned as the Celery task result.

- `/report/download/{task_id}` reads the cached task result and renders PDF via

ReportLab without re-running the LLM.

ReportData Contract

The PDF renderer accepts the current structured fields and keeps backward compatibility with older, shorter payloads:

Field	Type
`kategoria`	string
`streszczenie_wykonawcze`	string
`zakres_i_metodyka`	string
`wskazniki_liczbowe`	list of `{ nazwa, wartosc, jednostka }`

Field	Type
szczegolowa_analiza	list of strings
wdrozone_polityki_i_dzialania	list of strings
zidentyfikowane_ryzyka	list of strings
luki_w_danych	list of strings
rekomendacje	list of strings
zgodnosc_ze_standardami	list of strings
wnioski_i_zgodnosc_prawna	string

Security Notes

- Secrets live in `.env` and must not be committed.
- `JWT_SECRET` must be at least 32 characters.
- User-scoped API routes should use JWT-derived user identity where possible.
- Report history endpoints use JWT-derived user identity in the current code.
- Current hardening backlog: add backend admin-role enforcement to embedding reindex endpoints and add ownership verification to chat history retrieval.